

Multi-Agent LLM Orchestration for Embedded Systems Reliability Auditing: A Methodology and Case Study

Baxrom R. Reyimberganov^{1,*}

¹Department of Software Engineering, Urgench State University, Urgench, 220100, Uzbekistan

*Corresponding author: Baxrom R. Reyimberganov, Email: bahromreyimberganov0311@gmail.com, ORCID: [0009-0005-8124-0042](https://orcid.org/0009-0005-8124-0042)

Keywords: AI-assisted software engineering, large language model agents, multi-agent orchestration, verification gates, human-in-the-loop, embedded systems

Abstract

Large language model (LLM) coding agents are increasingly used not just to write code but to audit it — reading an existing codebase, finding defects, implementing fixes, and verifying them. This raises a methodological question distinct from “can an LLM write correct code”: how should such audits be structured so that their output is trustworthy, and where must human judgment remain load-bearing? We report on the methodology used to conduct the firmware reliability audit described in a companion case-study paper, in which sixteen defects across 32 embedded build configurations were found and fixed using a multi-agent LLM orchestration pattern: independent parallel agents scoped to non-overlapping files for each audit and fix pass, a compile-matrix verification gate that every change had to survive before being considered complete, and explicit human authorization at every escalation from read-only audit to code modification to production deployment. We report the methodology’s structure, quantify its verification coverage, and present two illustrative episodes in detail: a fix that was implemented, passed every static and compile-time check available, and was then deliberately reverted by human judgment because its correctness depended on a property (byte-range equivalence in a checksum computation) that could only be verified against physical hardware not available during the audit; and an incident in which a second, independent AI-assisted session operating on the same shared repository silently absorbed the audit’s in-progress, not-yet-authorized commits into its own unrelated commits, surfaced only by manual git-history inspection. We draw methodological lessons for practitioners structuring similar human–AI collaborative audits.

1 INTRODUCTION

The premise of this paper is narrower than “LLMs can write code” and more specific than “LLMs can find bugs”: it is about the *process* by which an LLM-based coding agent’s output becomes trustworthy enough to ship into a fleet of physical, field-deployed devices it will never itself observe running. This is a meaningfully harder bar than passing a code review, because embedded firmware for real hardware has a property much web or backend software does not: a large fraction of its failure modes are simply invisible to static analysis, unit tests, or even a clean compile, and only manifest against the real electrical and timing behavior of physical peripherals.

We report on the methodology used to conduct a systematic reliability audit of the firmware fleet described in a companion paper — 32 PlatformIO build configurations across six utility-sensor types —

in which an LLM coding agent, directed by a human engineer, found and fixed sixteen defects across four audit passes. Rather than repeating that paper’s defect-level findings, this paper asks: what *structure* did the audit itself have, what made its output trustworthy enough to deploy, and what went wrong in the process along the way?

Two process failures are reported here in detail because we consider them more broadly instructive than most of the individual code defects. First, one fix — a receive-side checksum verification, intended to reject corrupted serial-protocol frames — was implemented, and it passed every check available to the process: it compiled, it matched the surrounding code’s style and conventions, and static re-reading of the change found no fault with it. It was then reverted anyway, on the judgment that its correctness depended on a specific claim (that the byte range checksummed on receive exactly matched the byte range checksummed on transmit) that the audit’s available tools — a compiler and a careful human or AI reader — were fundamentally unable to verify, because verifying it required exercising the code against a real meter over a real cable. This is, we argue, the sharpest illustration available from this audit of where LLM-assisted verification’s ceiling actually is.

Second, partway through the audit, independent inspection of the repository’s git history revealed that a second AI-assisted session — working on an unrelated feature, in the same working directory, at the same time — had been running broad-scope commits (`git add -A` or equivalent) that silently absorbed the audit’s in-progress, not-yet-reviewed changes into its own commits, under unrelated commit messages, and had already pushed them to the shared remote. No data was lost and no incorrect code shipped as a result, but the incident surfaced a coordination failure category that is specific to multi-agent, multi-session AI-assisted development and that we believe is under-discussed in current practice: two independent agents, each individually well-behaved, can still interfere with each other purely through shared, un-versioned *intent* about who is allowed to commit what, when.

Our contributions are: (1) a description of a parallel-agent, scoped-file, compile-gated orchestration pattern for LLM-assisted code auditing, applied to an embedded systems case study; (2) a report of the pattern’s verification coverage and its growth over the course of the audit; (3) a detailed account of a conservatively-reverted fix as a worked example of the boundary between what static/compile-time verification can and cannot establish; and (4) a first-hand account of a concurrent-session git-coordination failure, offered as a concrete instance of a risk category we expect to recur as multi-agent AI-assisted development becomes more common.

2 METHODS

2.1 Orchestration pattern

The audit was structured as a sequence of phases, each following the same three-stage pattern, illustrated in Figure 1.

1. **Parallel, scoped audit.** For a given layer of the system (e.g., sensor calibration logic; RS-485 bus timing; ESP32 platform primitives), several agents were run concurrently, each assigned a disjoint subset of files or a distinct architectural concern, and each instructed to read full file contents rather than diffs and to report concrete, file-and-line-cited findings rather than general impressions. Running agents in parallel over disjoint scope, rather than one agent sequentially over the whole codebase, was a deliberate choice: it bounds each agent’s context to what it can plausibly reason about completely, and it makes the audit’s total wall-clock cost roughly independent of how many subsystems are reviewed.
2. **Human synthesis and authorization.** Findings from all parallel agents were consolidated, prioritized by the human engineer, and explicitly scoped before any code was modified — including, in

one documented instance (Article 2, § 2.5’s checksum fix), an explicit human instruction to exclude a specific, already-identified finding from the authorized fix set pending separate risk judgment.

3. **Parallel, scoped fix, then a compile-matrix gate.** Once a fix set was authorized, implementation again used parallel agents with disjoint file ownership (to avoid concurrent-edit conflicts on the same file), followed by a dedicated verification pass: a full recompilation of every PlatformIO build configuration plausibly affected by the change set. A fix was not considered complete until this gate passed; a gate failure routed back to fix implementation rather than being treated as acceptable residual risk.

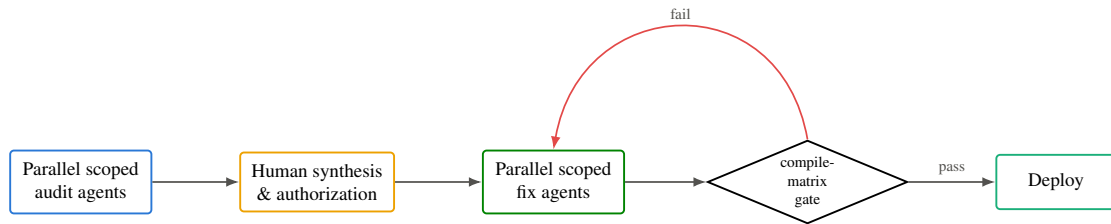


Figure 1: The three-stage orchestration pattern repeated across each audit phase: parallel scoped audit agents feed a human synthesis/authorization step, which scopes a parallel scoped fix pass, gated by full compile-matrix recompilation before deployment; a gate failure routes back to fix implementation rather than being accepted as residual risk.

2.2 Verification coverage

The compile-matrix gate’s coverage was not static: it grew over the course of the audit as previously-uncovered build configurations were identified as deserving verification. Figure 2 reports this growth across three points: the audit’s starting continuous- integration matrix (8 build configurations, chosen as one representative target per utility-sensor type); an intermediate expansion after the audit identified that two additional build configurations, originally treated as bench-test-only, were in fact used as passively field-deployed diagnostic tools and therefore deserved the same regression protection as production targets; and the full verification gate applied to the audit’s final phase, which recompiled fifteen build configurations — including every diagnostic/bring-up mode touched by that phase’s changes — before any fix in that phase was considered complete.

2.3 A formal decision rule for the reverted fix

The checksum fix reverted in Article 2 § 2.5 is worth formalizing, because the decision to revert was not ad hoc: it follows from a simple expected-cost argument available whenever a fix’s correctness depends on a claim that static verification cannot settle. Let p be the (unknown, unmeasurable-by-static-means) probability that the fix’s critical claim holds — here, that the transmit- and receive-side checksummed byte ranges are exactly equivalent. Let C_{FP} be the cost incurred if the fix ships and the claim is *false* (every real hardware read is silently rejected as corrupted — a fleet-wide failure of the platform’s core metering function), and C_{FN} the cost of reverting when the claim was in fact *true* (foregone protection against the comparatively rare event of genuine line-noise corruption, which the pre-existing, field-proven behavior had already been operating under). Shipping is only justified in expectation when

$$p C_{FN} > (1 - p) C_{FP} \iff p > \theta = \frac{C_{FP}}{C_{FP} + C_{FN}}. \quad (1)$$

Here $C_{FP} \gg C_{FN}$ — a fleet-wide loss of the core metering function dominates a missed opportunity to catch rare noise — so $\theta \rightarrow 1$: shipping is only justified at very high confidence. Critically, the audit’s

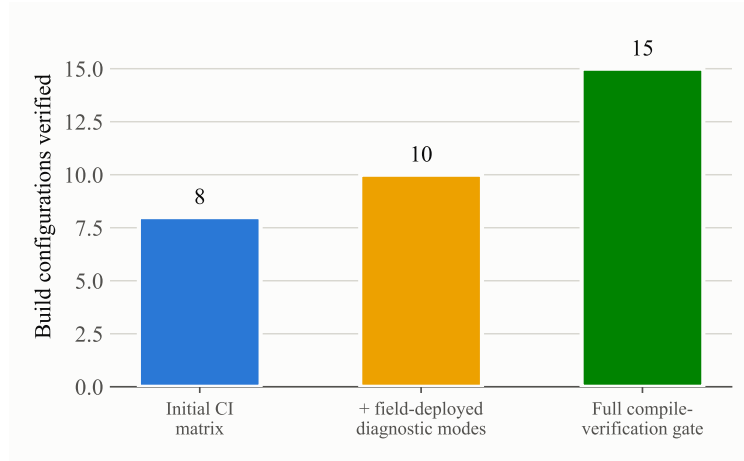


Figure 2: Growth of compile-verification coverage over the audit. The matrix widened as the audit itself discovered previously-uncovered, field-relevant build configurations — the verification net grew because the audit’s own findings changed what counted as “in scope,” not because of a plan fixed in advance.

available verification tools (a compiler; a careful re-reading of the transmit- and receive-side byte- range arithmetic) could establish internal consistency but could not establish p close enough to 1 to clear a threshold that high, because the claim is fundamentally about *runtime behavior of physical hardware*, not about the code’s static structure. Eq. (1) is offered as a reusable framing for similar decisions: when a fix’s risk asymmetry is large and its critical correctness claim is outside what your available verification method can settle, the correct default is to not ship, regardless of how confident static inspection feels.

2.4 The concurrent-session incident

Approximately midway through the audit, a routine pre-commit check (`git fetch` plus a diff against the remote) surfaced that the working directory’s git history contained commits the audit had not authored, under commit messages describing an unrelated feature (a database schema migration), whose diffs nonetheless included the audit’s own in-progress, not-yet-authorized changes to CI/CD configuration and firmware files. Reconstructing the timeline from commit timestamps and file diffs established that a second AI-assisted session, working concurrently on the same repository checkout for an unrelated purpose, had run broad-scope commits (consistent with `git add -A` or equivalent) that captured whatever was in the working tree at that moment — including the audit’s uncommitted edits — and pushed the result to the shared remote, all without any signal visible to either session that the other was active.

No commit was lost, no incorrect code reached production as a direct result (the swept-in commits were independently re-verified before being trusted), and the incident was resolved by direct communication with the repository owner. We report it not as a failure of either individual session’s behavior — both, examined separately, behaved reasonably given their own instructions — but as a coordination gap that is specific to this development mode: two agents, each respecting the specific files and commits they were told about, had no shared mechanism for learning about each other’s existence at all.

3 RESULTS

Table 1 summarizes the audit’s process-level metrics, distinct from the defect-level results already reported in the companion paper.

The zero-regressions result is a genuine finding, not a tautology: the compile gate recompiles every targeted build configuration from source after every change, so a regression would manifest as a build failure requiring the same fix-and-re-verify loop shown in Figure 1’s feedback edge. That loop was never

Metric	Value
Audit/fix phases (parallel-agent orchestrated)	3, each following the pattern in Figure 1
Build configurations in final compile gate	15, up from an initial 8
Defects found and fixed (companion paper)	16, across 4 defect classes
Fixes reverted after passing every static/compile check	1, documented in § 2.3
Regressions introduced across all fixes (per the compile gate)	0
Pre-existing, audit-unrelated defect caught incidentally by the compile gate	1 (a latent function-signature mismatch left over from an earlier, unrelated refactor, in a build configuration the gate happened to cover)

Table 1: Process-level metrics for the three orchestrated audit/fix phases, as distinct from the defect-level results reported in the companion case-study paper.

exercised by an audit-introduced regression across sixteen fixes; it was exercised once, but by a defect the audit did not introduce — the latent, pre-existing signature mismatch noted in Table 1 — which the gate caught only because the affected build configuration happened to be within that phase’s verification scope, itself a direct consequence of the coverage growth reported in Figure 2.

4 DISCUSSION

A compile gate is necessary but its ceiling is lower than it feels. The zero-regression record across sixteen fixes is genuine evidence that a parallel-agent, scoped-file, compile-gated pattern can produce a large volume of verified change without introducing new breakage. But the reverted fix in § 2.3 is the sharper result: that fix cleared every check the process had, including a full, green compile-matrix run, and was wrong to ship anyway, because its critical correctness claim was never the kind of claim a compiler — or a careful reader, AI or human — can settle. We think this is the central methodological lesson of the audit: a compile-gated verification process should be read as bounding *one class* of risk (does the code build and does its logic hold up to inspection) and explicitly not as bounding risk that depends on real-world, physical, or runtime properties outside that method’s reach. Eq. (1) is offered as a way to make that boundary an explicit part of the process, rather than a judgment call made silently and inconsistently fix-by-fix.

Human authorization at phase boundaries did meaningful work. Every transition from read-only audit to code modification, and from code modification to the specific act of pushing to a shared remote, was gated on an explicit human decision in this audit — including, notably, the human overriding an agent’s completed, verified work in the reverted- fix case. We do not have a controlled comparison against an audit without this gating, but we note that the one case where the process’s own verification (compile-matrix) was insufficient was also exactly the case where human judgment, not tooling, supplied the missing signal (recognizing that the claim at stake was unverifiable by the available method at all, not merely unverified so far). This suggests human authorization is most valuable not as a rubber stamp on agent output generally, but specifically at the boundary of what the process’s own verification method can reach.

Concurrent AI-assisted sessions need an explicit coordination mechanism, not just good individual behavior. The git-collision incident in § 2.4 was not caused by either session behaving incorrectly relative to its own instructions; it was caused by the absence of any shared signal that a second session existed at all. As LLM-assisted development sessions become cheaper to run concurrently, we expect

this failure mode to recur, and we do not think it is solved by more careful individual agent behavior — it requires an explicit protocol (e.g., a session-visible lock, a shared status log, or a policy against broad-scope commits from automated sessions) external to any single agent’s own good judgment.

Limitations. This is a single case study of one audit conducted by one human–AI pairing on one codebase; we make no claim that the specific numbers reported here (three phases, fifteen build configurations, one reverted fix) generalize quantitatively to other projects. The process-level claims — that scoped parallel agents plus a compile gate plus human boundary authorization produced zero audit-introduced regressions — are also self-reported by the same process being evaluated, without an independent auditor checking the auditors; we consider external replication, ideally by a team not otherwise involved in the codebase, the natural next step before treating this pattern as validated rather than promising.